



The Application of AI to Chip Design

Haoxing (Mark) Ren, Director of Design Automation Research, NVIDIA

AI for Chip Design Research @ NVIDIA

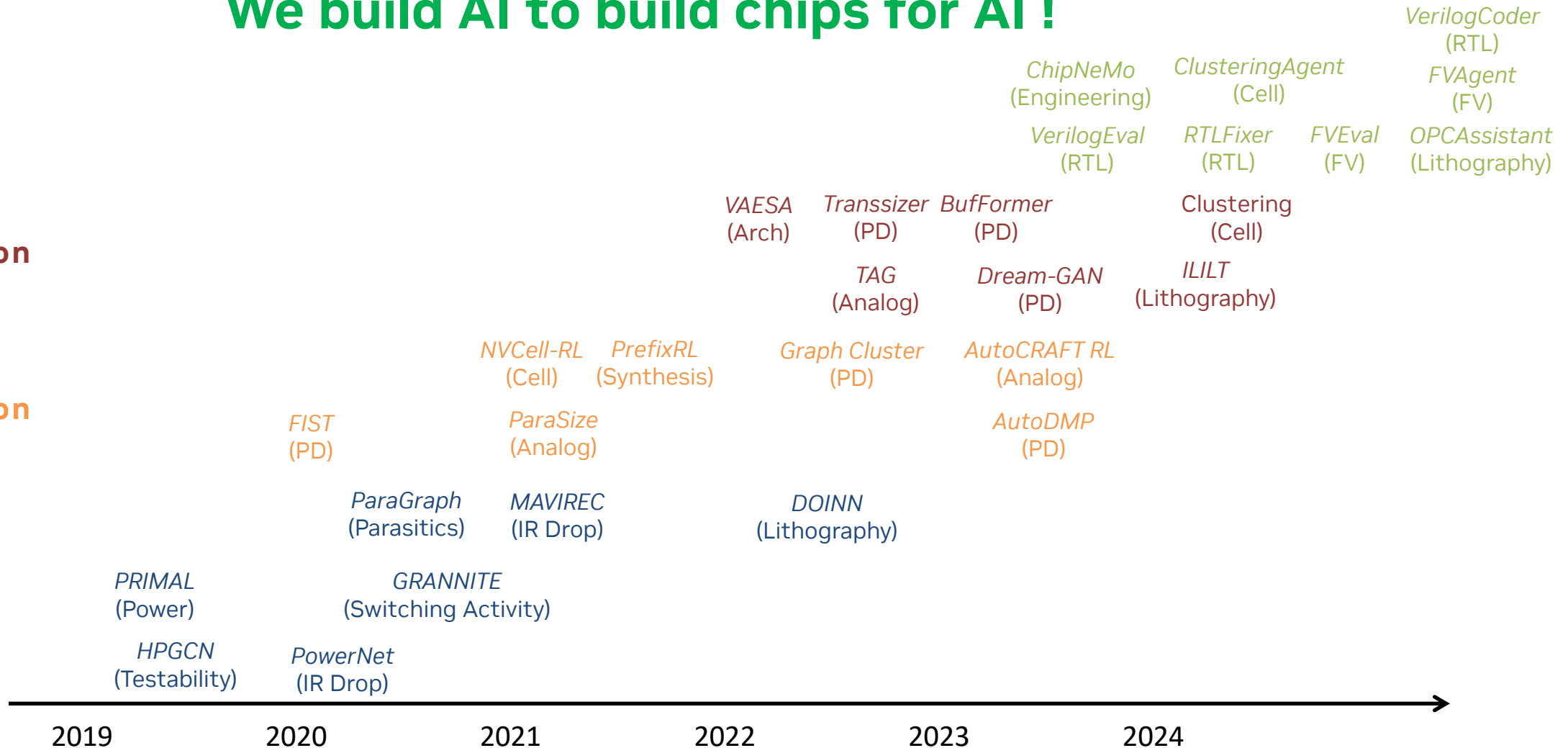
We build AI to build chips for AI !

Design
Assistance

Design
Optimization
(Gen AI)

Design
Optimization
(RL, BO)

Design
Analysis

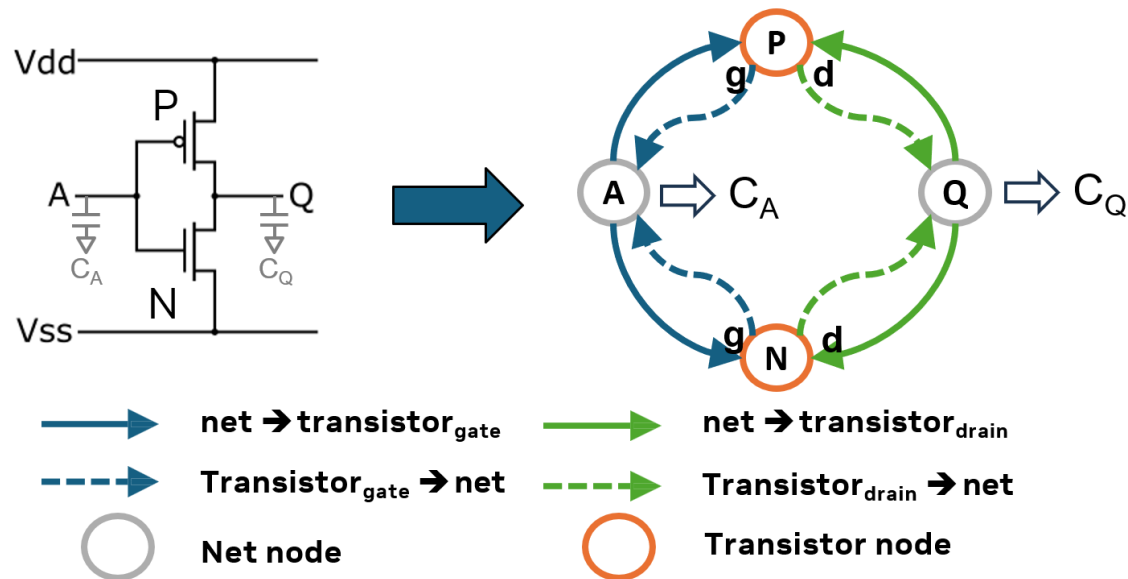


Analysis – Parasitics Prediction

Impact of layout parasitics on schematic design

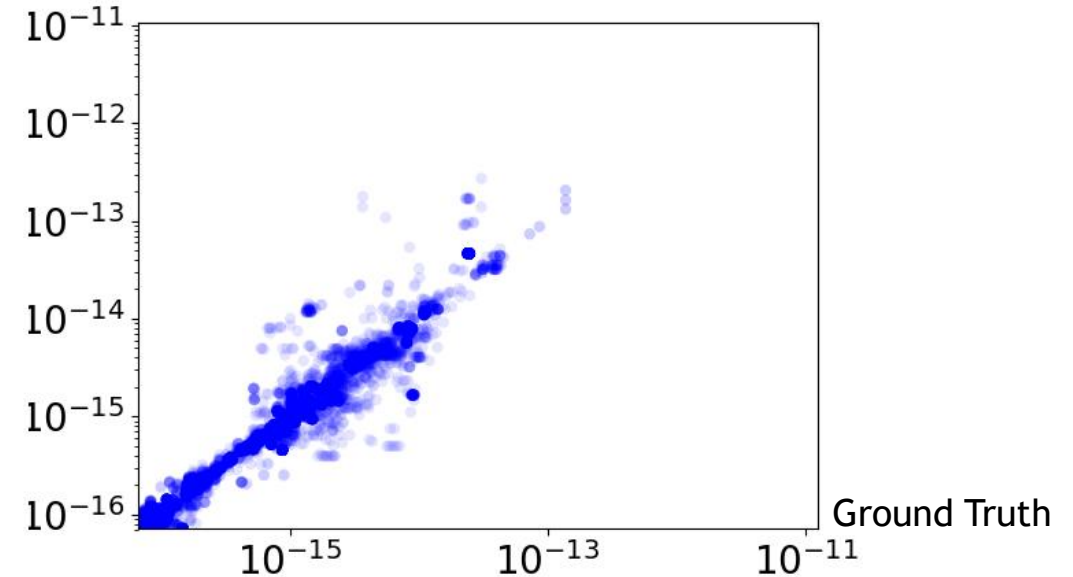
Use AI to predict layout parasitics from schematic

Convert schematic to graph and learn with GNN



Circuit Schematics to Heterogenous Graph Conversion

Cap Prediction (F)



MAE=0.852fF MAPE=15%
Simulation error reduced to <10%

Optimization – Macro Placement

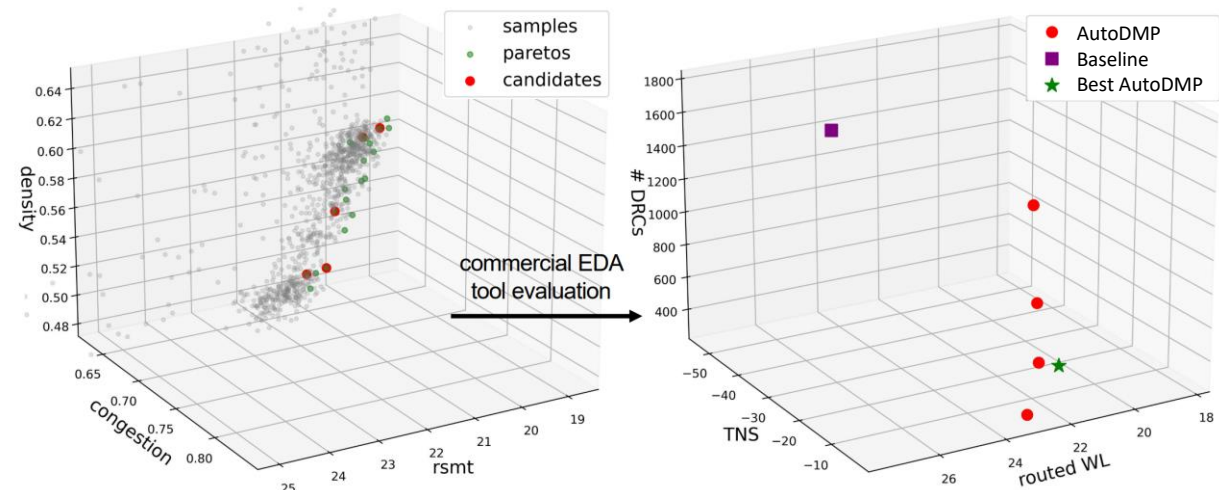
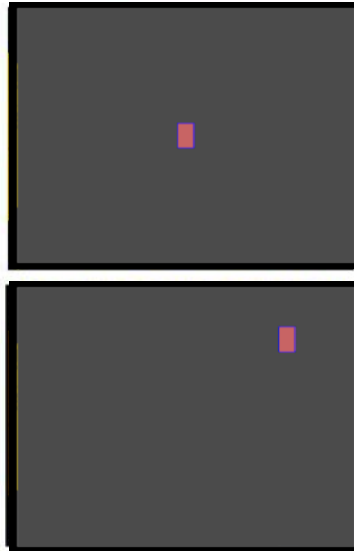
Macro placement quality is very important for physical design

Placement parameters have a huge impact on macro placement

Multi-objective Bayesian optimization: wirelength, congestion, density

Find better macro placement with open-source GPU accelerated placement tools

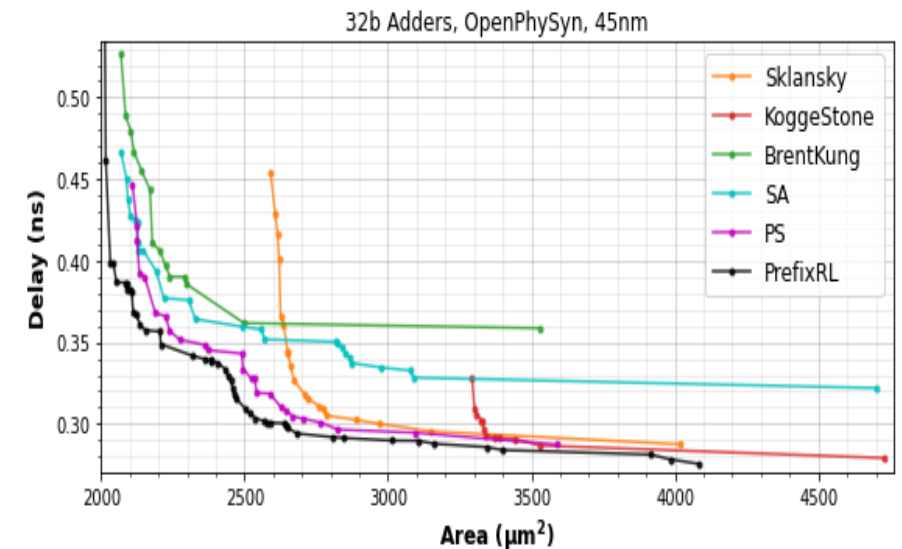
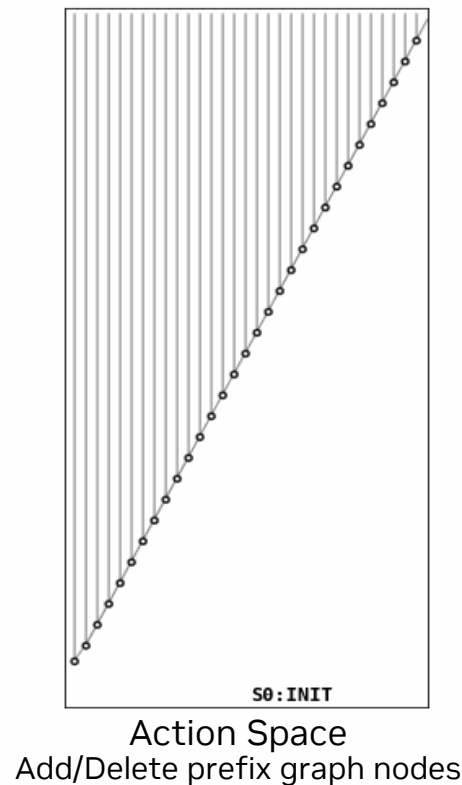
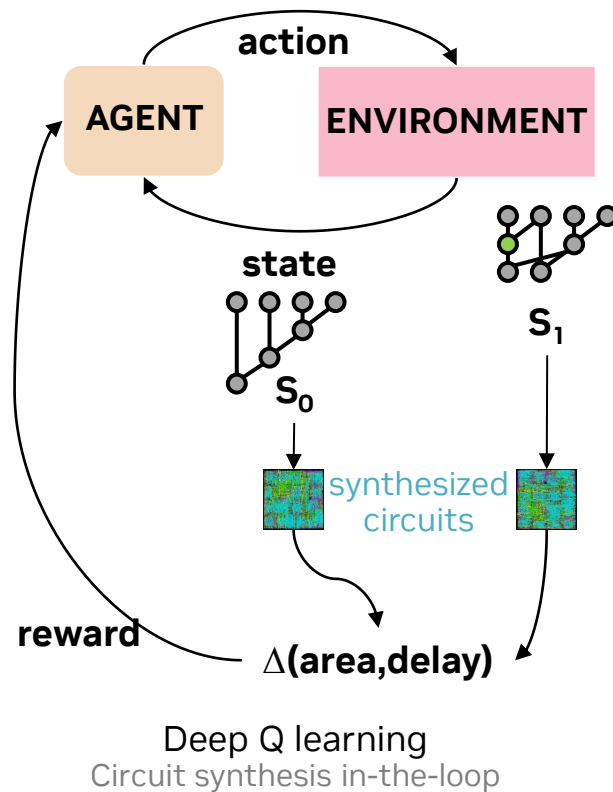
Parameter	Search Range
*horiz. initial position	[0.2, 0.8] (%)
*vert. initial position	[0.2, 0.8] (%)
*horiz. macro halo	technology dep.
*vert. macro halo	technology dep.
target density d_{target}	$[a_{\text{util}} - 0.2, a_{\text{util}}]$ (%)
density weight	$[1e^{-6}, 1.0]$
smooth HPWL model	{LSE, WA}
smooth HPWL initial γ_0	[0.10, 0.50]
GD initial LR lr_0	$[1e^{-4}, 1e^{-2}]$
GD LR decay	[0.99, 1.0]
GD optimizer	{Adam, Nesterov}
# horiz. global bins	{256, 512, 1024, 2048}
# vert. global bins	{256, 512, 1024, 2048}
λ update lower coeff. L	[0.90, 0.99]
λ update upper coeff. U	[1.01, 1.15]
λ update Δ HPWL _{REF}	$[1.5e^5, 5.5e^5]$



Optimization - Design Better Datapath

Datapath synthesis important for GPU

Optimize prefix adder structure with RL

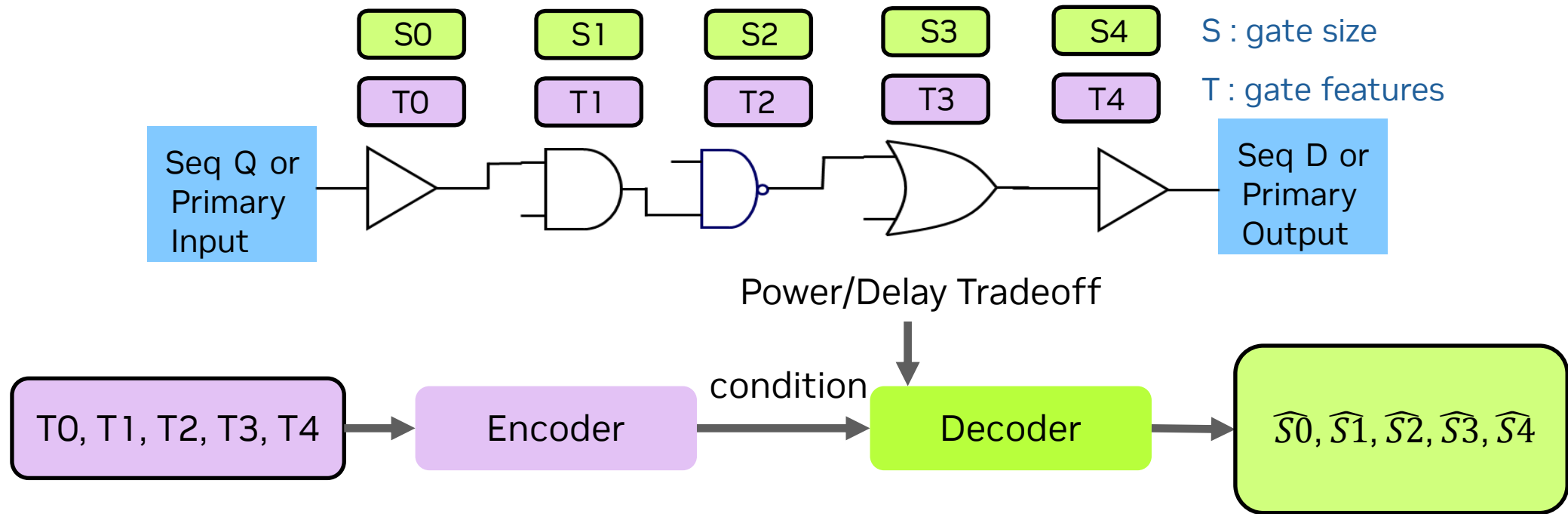


PrefixRL achieves better results than well known adder architectures

Optimization - Generate Optimal Gate Size

Timing/power optimization such as gate sizing affects scalability of PD tools

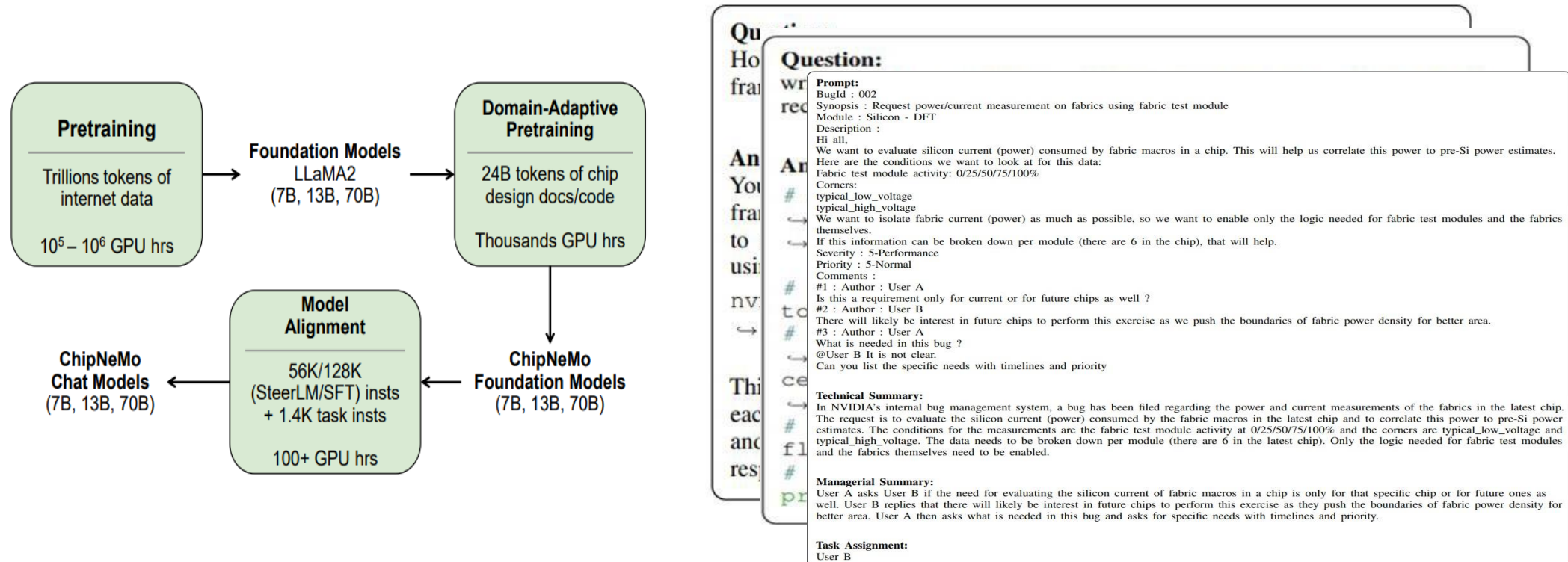
Model a path of gates as a sequence, generate optimized gate sizes using Transformer



100X – 1000X speedup compared to traditional optimization with similar PPA

Assistance - ChipNeMo

Answer questions about designs, infrastructures, tools, flows, HW domains, etc.
Generate scripts for specific tasks (VLSI)
Summarize bug reports, predict task assignment



Challenges

Analysis

“Inaccurate predictions”

Optimization:

“AI is not as good as existing algorithms”

“AI will never get better results than the data it trained on”

Assistance

“Hallucinations”

“Can not solve complex problems”

Compare AI and Algorithm

	Algorithm	AI
Categories	Placement, route, synthesis, CTS, etc	Supervised, unsupervised, reinforcement learnings, etc
Optimality	Known	Unknown
Robustness	Works on any data distribution	Do not work if training and inference distribution mismatch
Training	No	Require a lot data
Interpretability	Behavior explainable	Not explainable
Pros	Solve a known problem efficiently	Solve any problem by learning from complex data
Cons	Oversimplification of dynamic, complex problem	Difficult to leverage the mechanics of the problem

Similar Discussions

Two paradigms for Artificial Intelligence

The logic-inspired approach

The essence of intelligence is using symbolic rules to manipulate symbolic expressions.

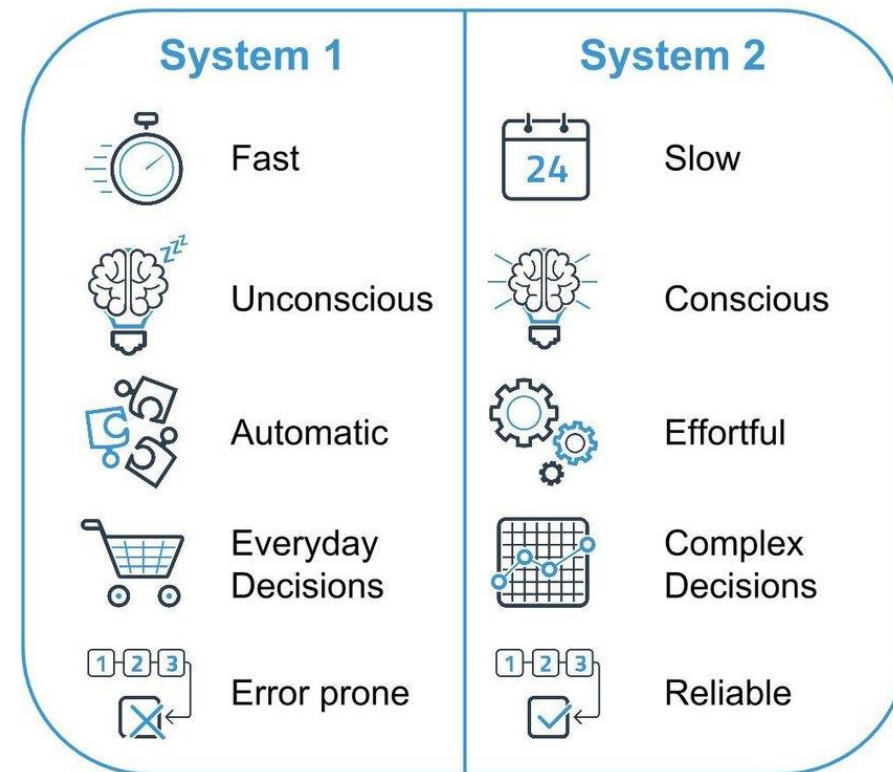
We should focus on reasoning.

The biologically-inspired approach

The essence of intelligence is learning the strengths of the connections in a neural network.

We should focus on learning and perception.

Jeff Hinton: Turing Award Lecture

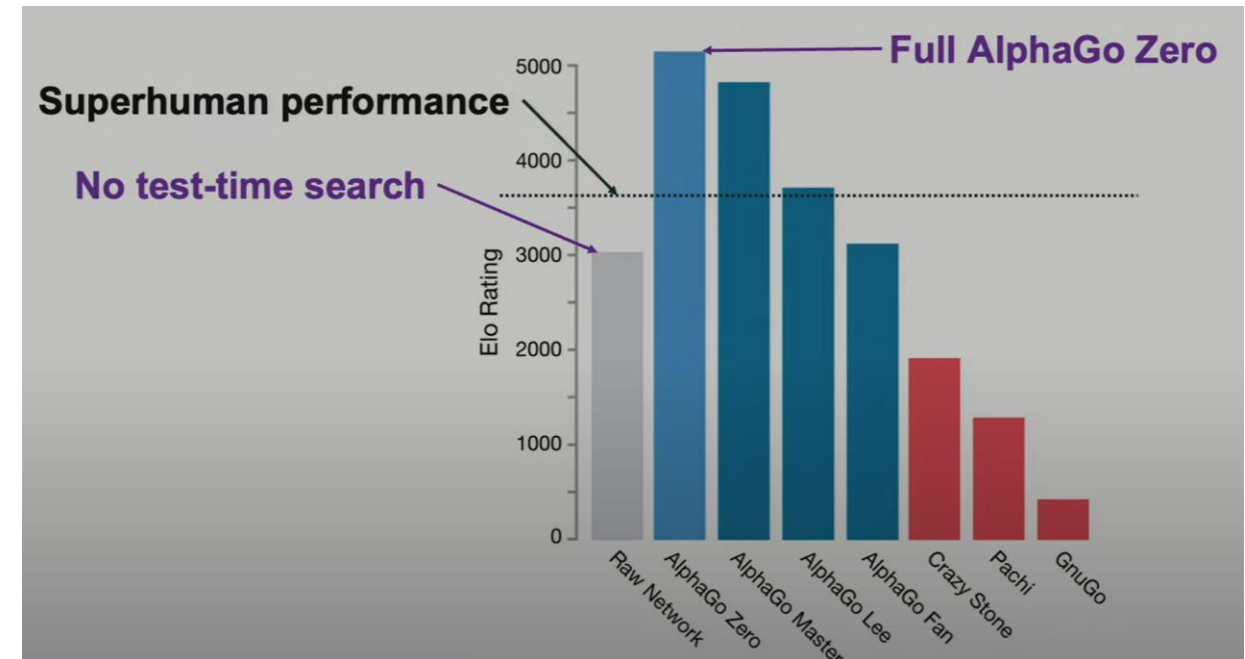
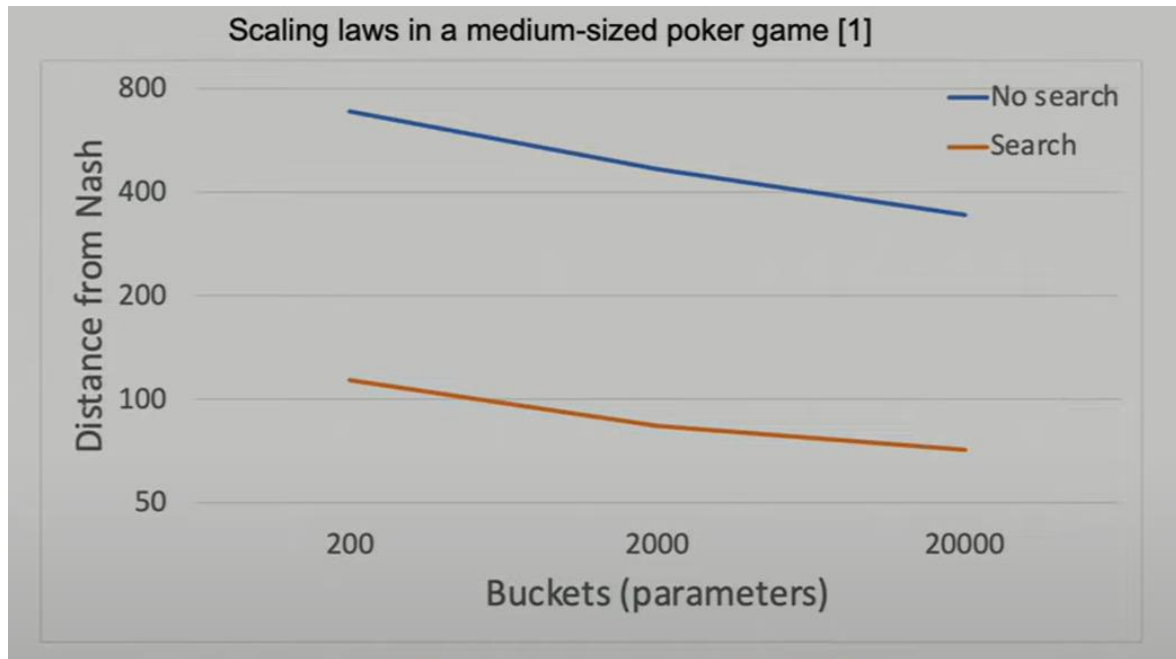


Daniel Kahneman: Thinking, Fast and Slow

Integrate AI with the Algorithm @ Inference

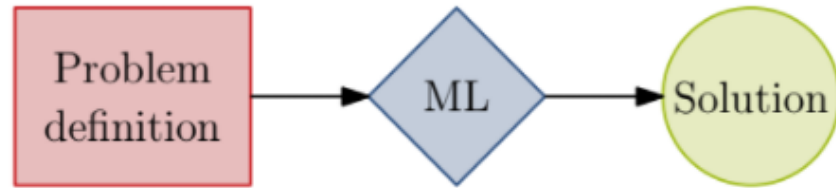
Example – Gaming

Search helps AI game model perform much better

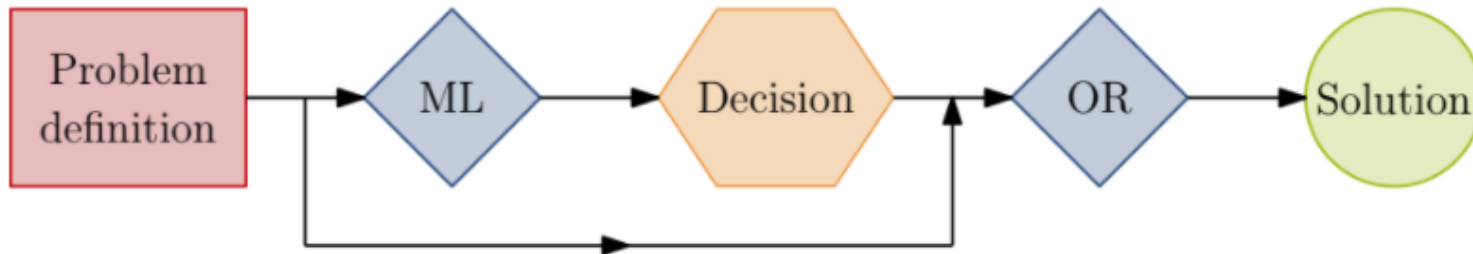


Noam Brown: Parables on the Power of Planning in AI: From Poker to Diplomacy

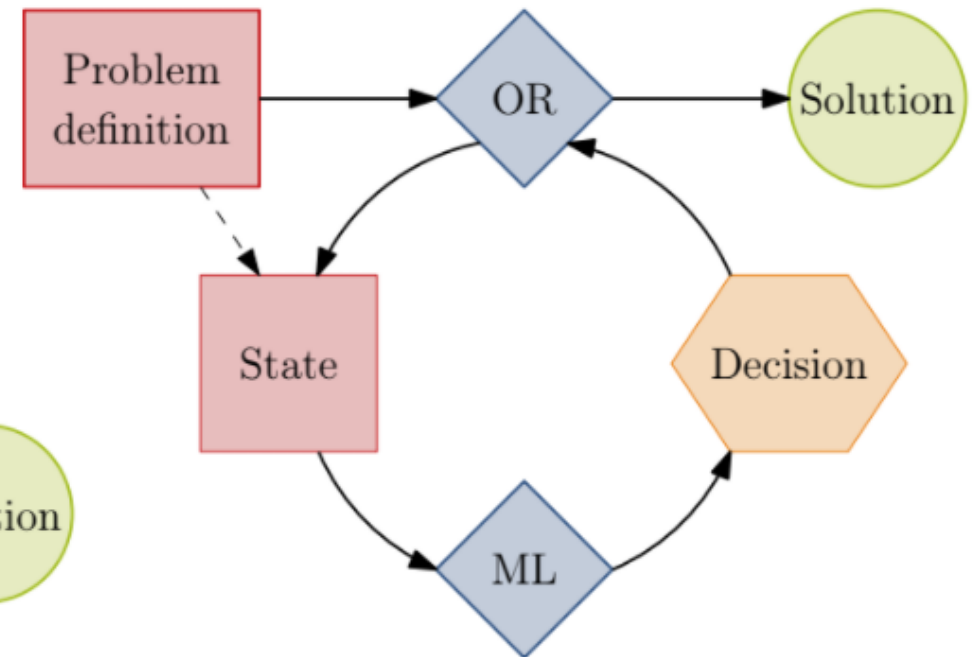
ML for Combinatorial Optimization



End-to-end learning



Coarse grain integration



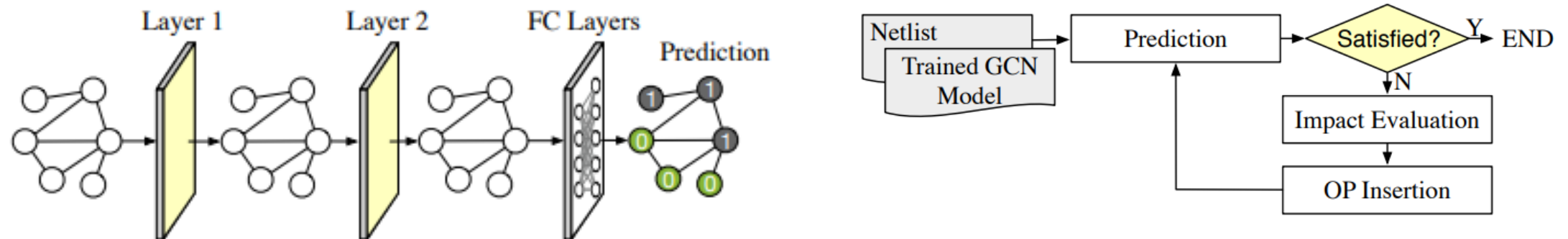
Fine grain integration

TPI: AI as Oracle for Algorithm

GCN model predicts node testability: Difficult-to-test nodes (DTN) prediction

Model acts as a predictor/oracle for a test point insertion (TPI) algorithm

Although model accuracy is only 90%, reduced test points by 11% and test patterns by 6% to achieve similar coverage as a commercial test point insertion algorithm



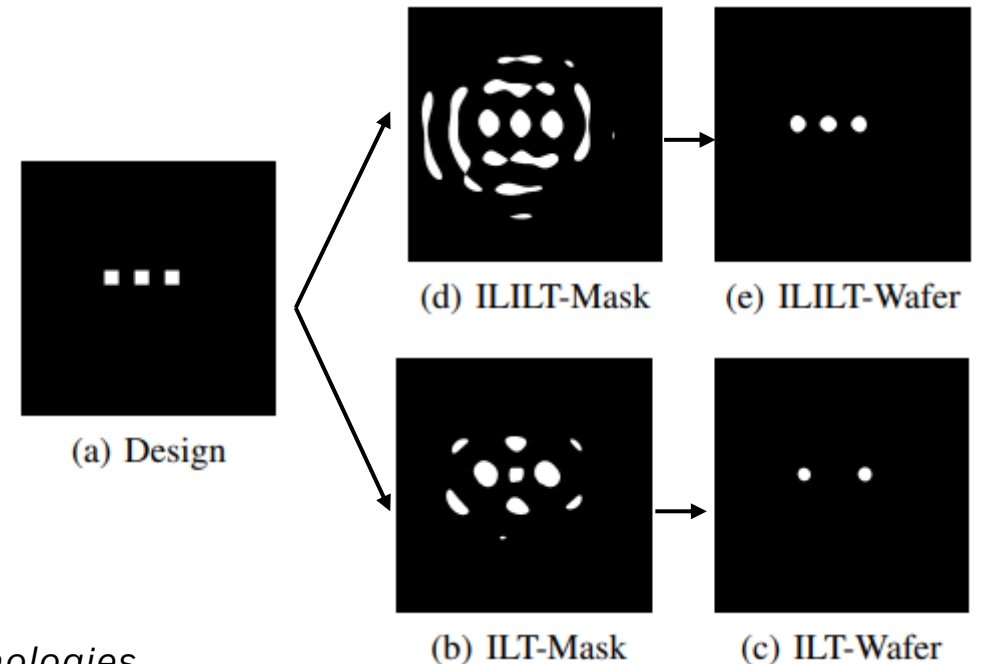
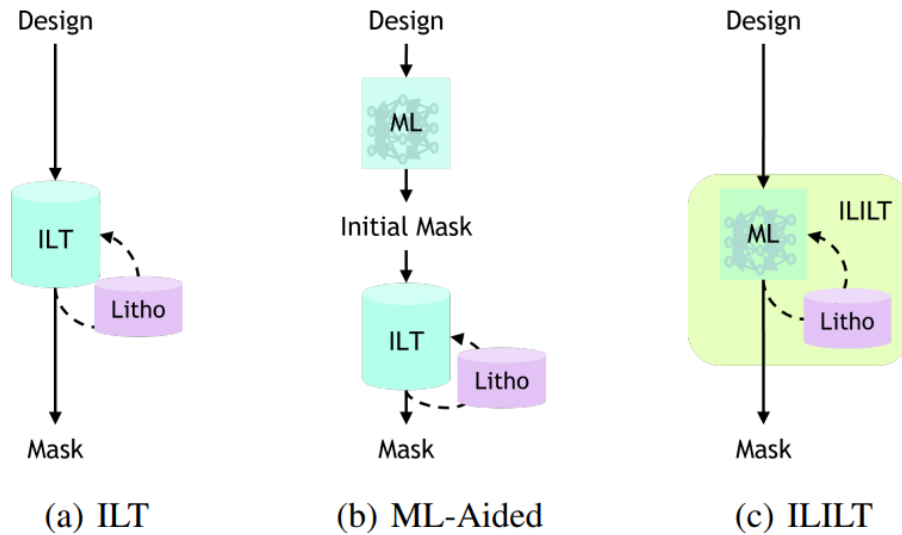
ILILT : Ground AI with Algorithm

Learn the optimized mask from ILT (inverse lithography Technology) solver

Use Litho forward simulation algorithm to ground the model

Increase inference time compute with multi-step inference

Better quality than ILT (generated the training data) with 10X speedup



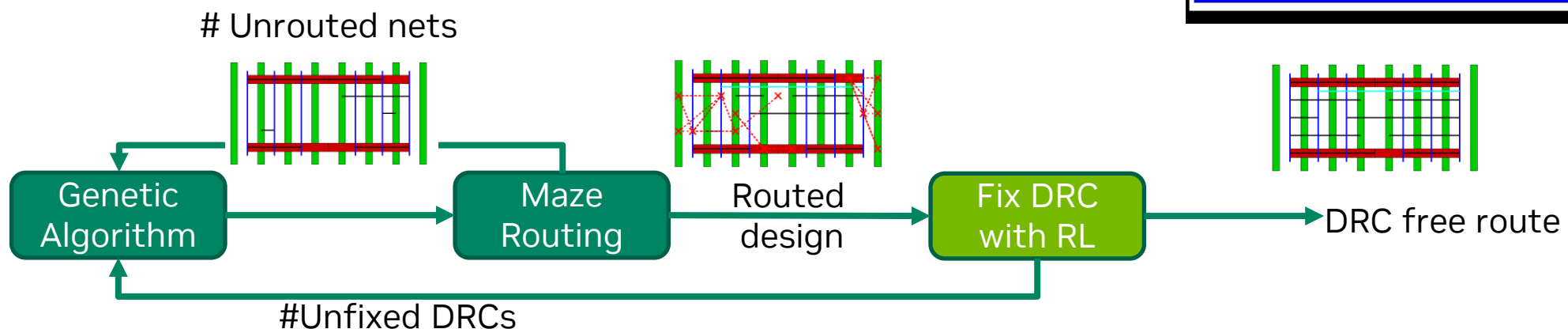
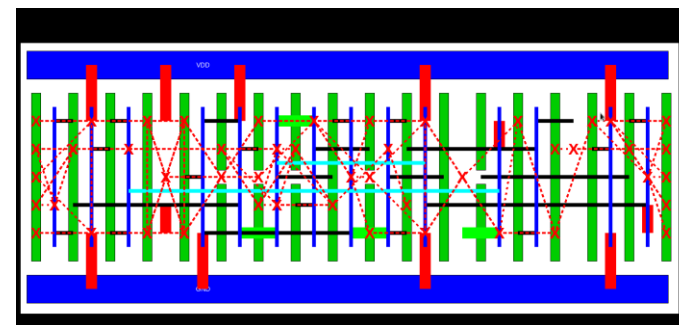
H. Yang et al, ILILT: Implicit Learning of Inverse Lithography Technologies

NVCell: AI as Part of the Algorithm

Auto routing for std cells, overcoming DRC challenge

Use an algorithm to route, but RL to fix DRC

Generate std cell layout of entire industrial libraries with better quality than designer



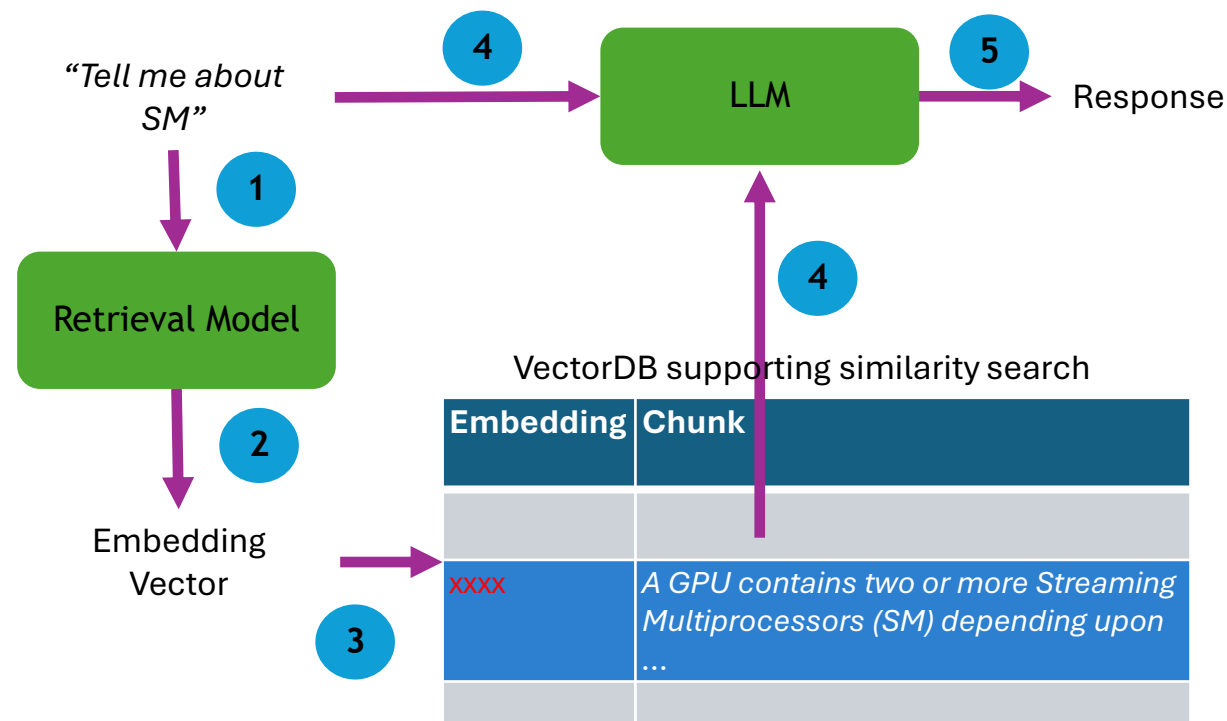
RAG : Algorithm @ Inference

Retrieval Augmented Generation (RAG) helps to ground LLM with curated documents

Help reduce **hallucination**

Incorrect, outdated, conflicting, scarcity of the training data

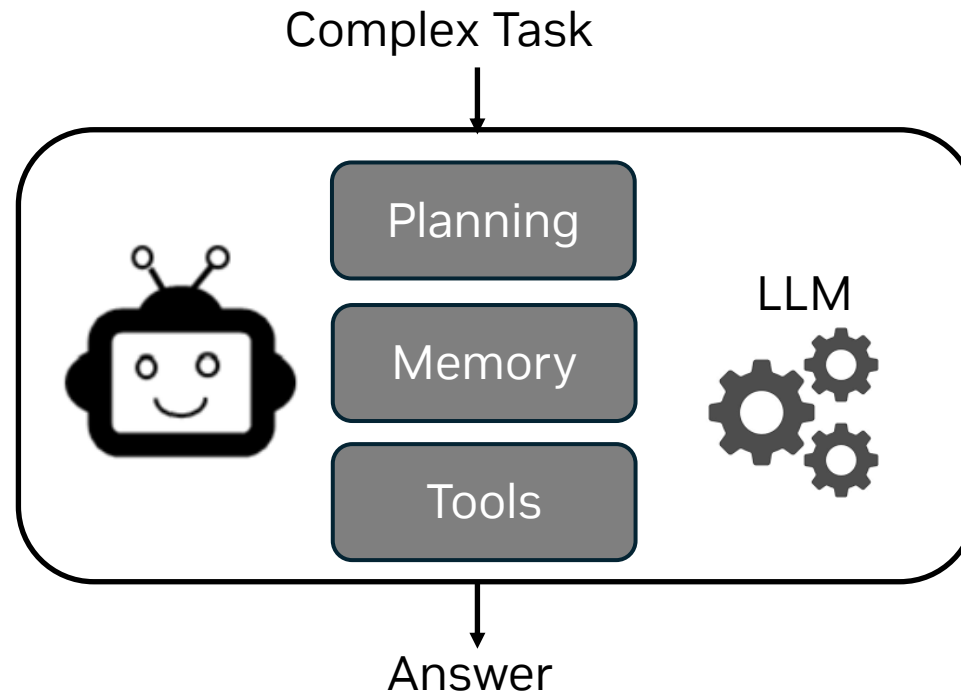
RAG leverages the search algorithm (embedding similarity “Hash”) to retrieve the relevant chunks as context for the question



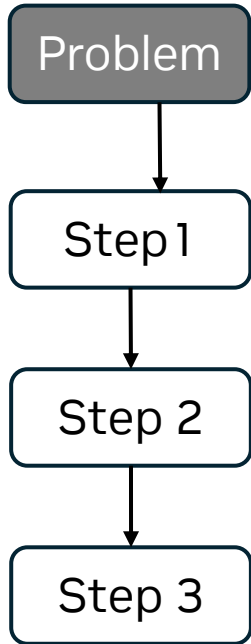
RAG Illustration

Agentic System: Algorithm @Inference

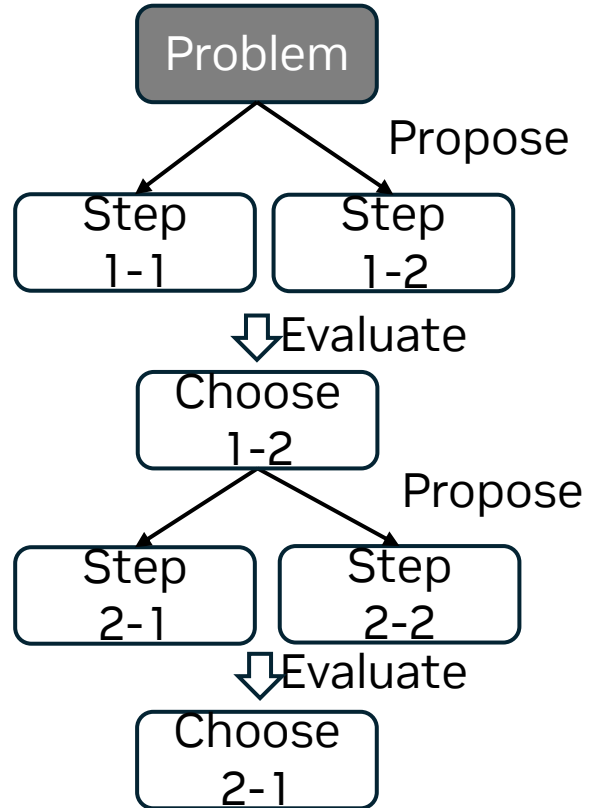
Problem decomposition enables Agent to handle complex tasks
Agent uses algorithms with LLM as compute unit



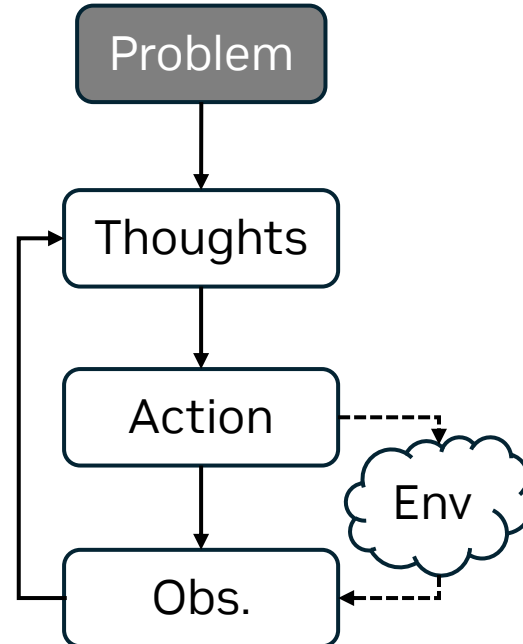
Planning



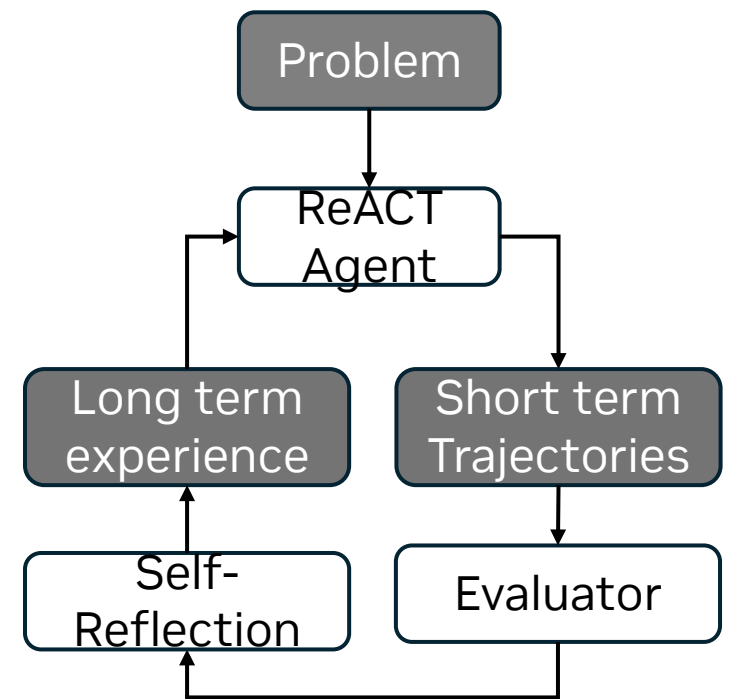
Chain-of-Thought
(Procedure)



Tree-of-Thought
(Branch)



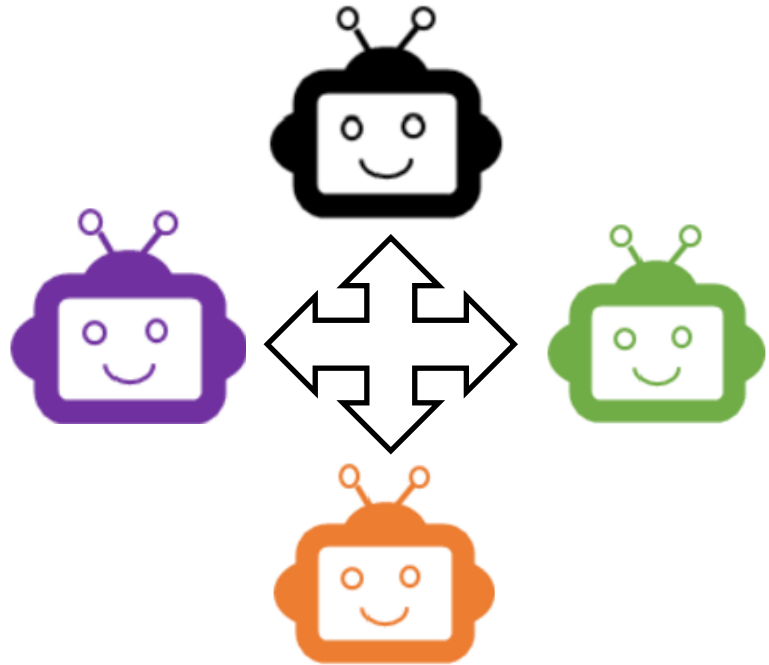
ReACT
(Loop)



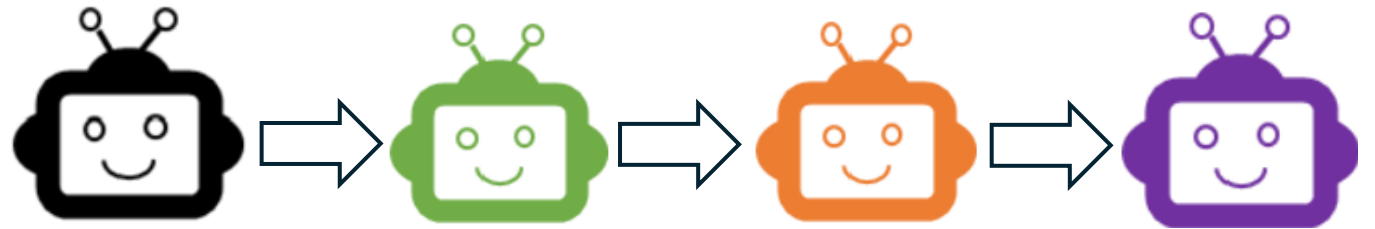
Self-Reflection
(Multi-Level Loop)

Multi-Agent

Hierarchical decomposition with dynamic and static structures



Agent Conversation
(dynamic)



Task Flow
(static)

Timing Report Analysis

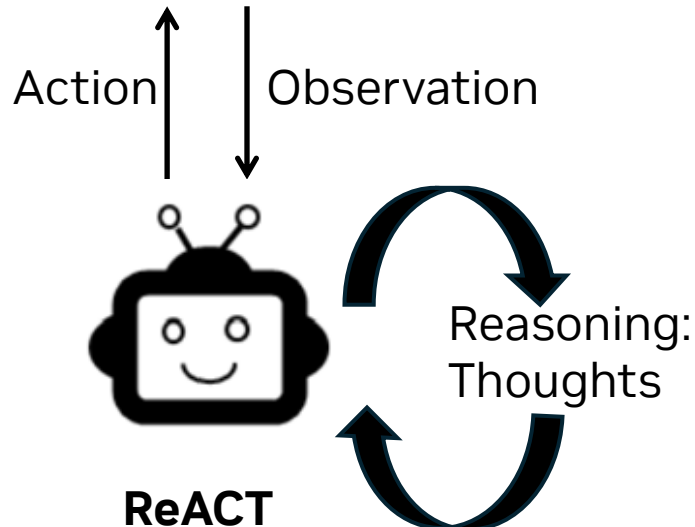
Tools

timing_metric_calculation_tool:

Calculate changes in WNS, TNS, or FEP

slack_distribution_calculation_tool:

Calculate changes in the slack distribution



CoT prompting

Let's think step by step.

Firstly, analyze the change of WNS, and TNS for all designs in all PVT corners tables of these two "datecode" settings and explain with numbers using tools.

Secondly, comparing "FEP" of two datecode settings in all PVT corners tables.

Thirdly, analyze the slack distribution to identify the distribution of "slack less than 0" paths.

Finally, summarize the analysis in the following aspects:

1. Provide key takeaways, comparison, and suggestions with bullet points of the two "datecode" settings based on previous steps.
2. Identify the corner which still suffers many timing violations if any.

You need to use the **provided tools** to analyze the timing metrics! You are not good at math!

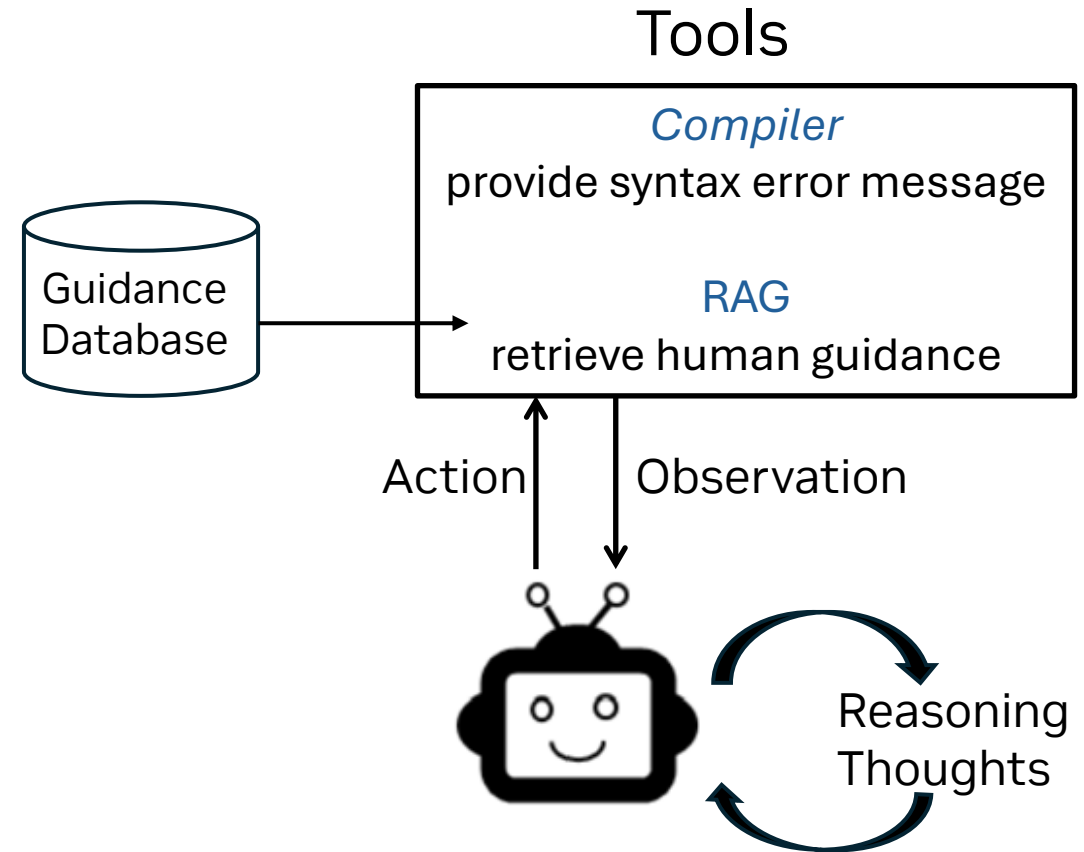
RTLFixer : Fix Syntax Error in RTL code

55% of GPT-3.5 Verilog generation errors are Syntax errors

Leverage **ReAct** framework to drive Verilog compiler to get feedback

Leverage **RAG** to retrieve human guidance for each syntax error

Fixing 99% of Syntax Errors on VerilogEval-syntax Benchmark

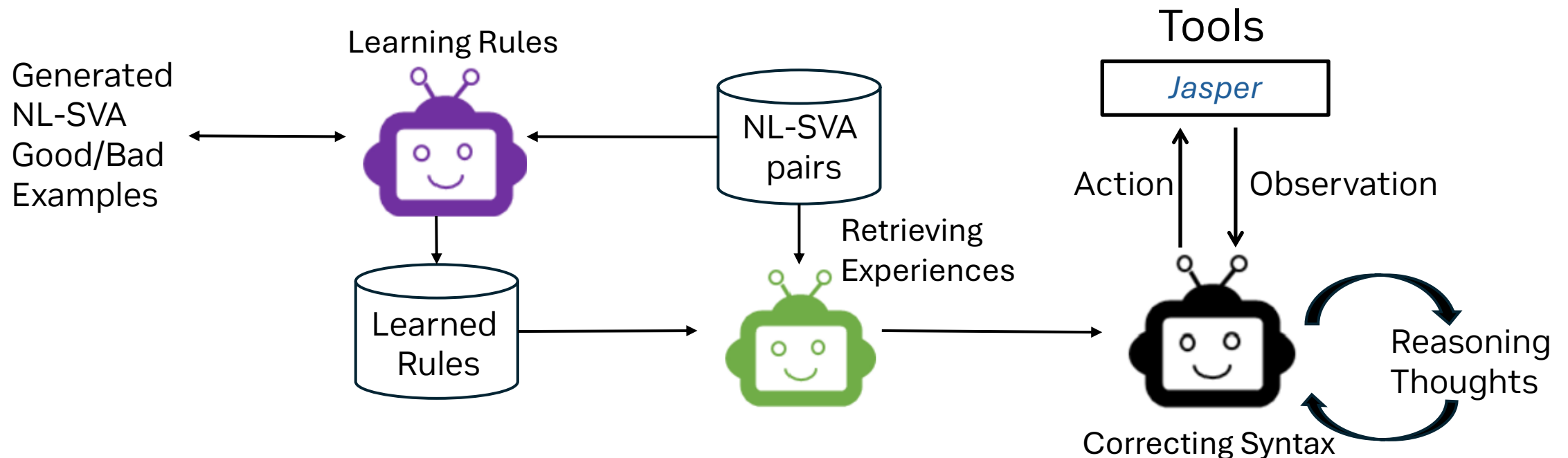


FVAgent : Generate SVA From NL

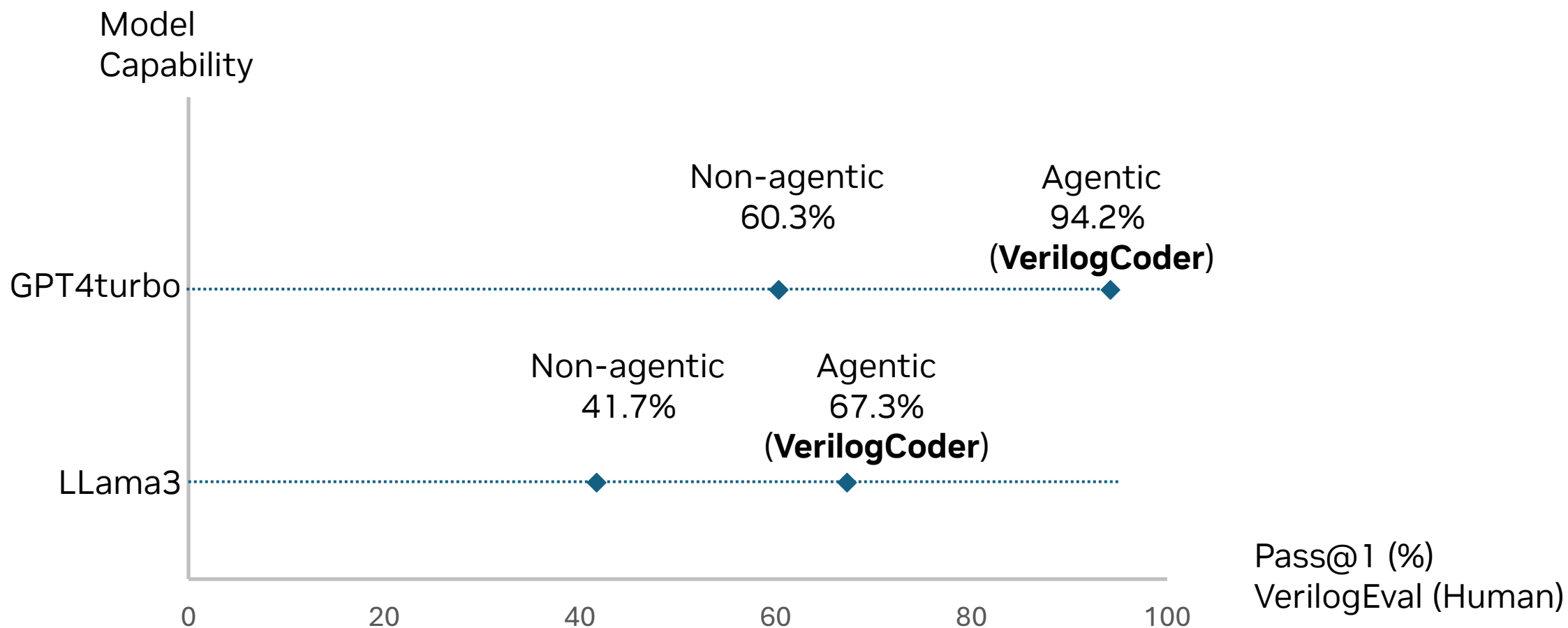
Self-Learning: Generate rule knowledge base for RAG

Multi-Agent task flow : Retrieval: prior NL-SVA pairs as well as learned rules; Correct syntax errors with tools

Improve Syntax/Function corrections on all models



VerilogCoder Agent



VerilogCoder Agent

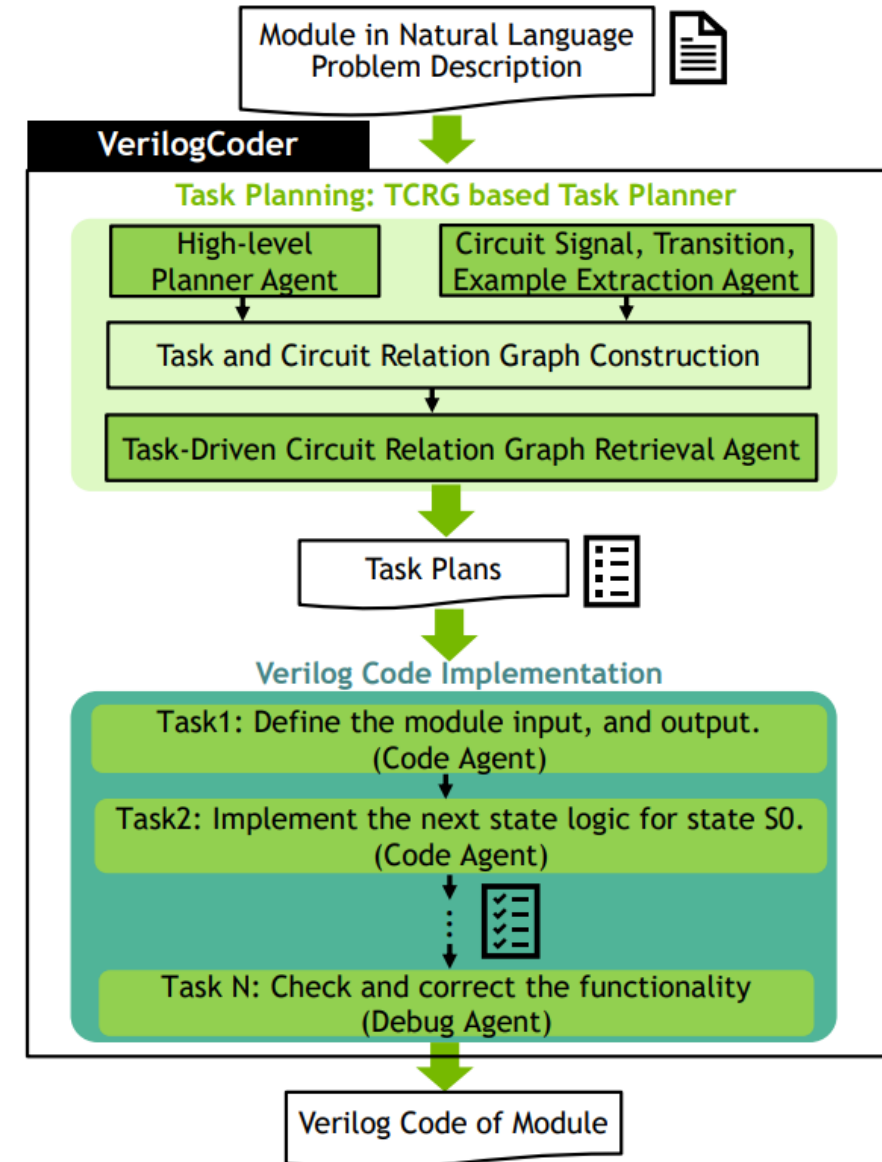
Dynamic task planning and execution
Tasks executed by single or multi-agents
Special knowledge base and tools

Task Planning

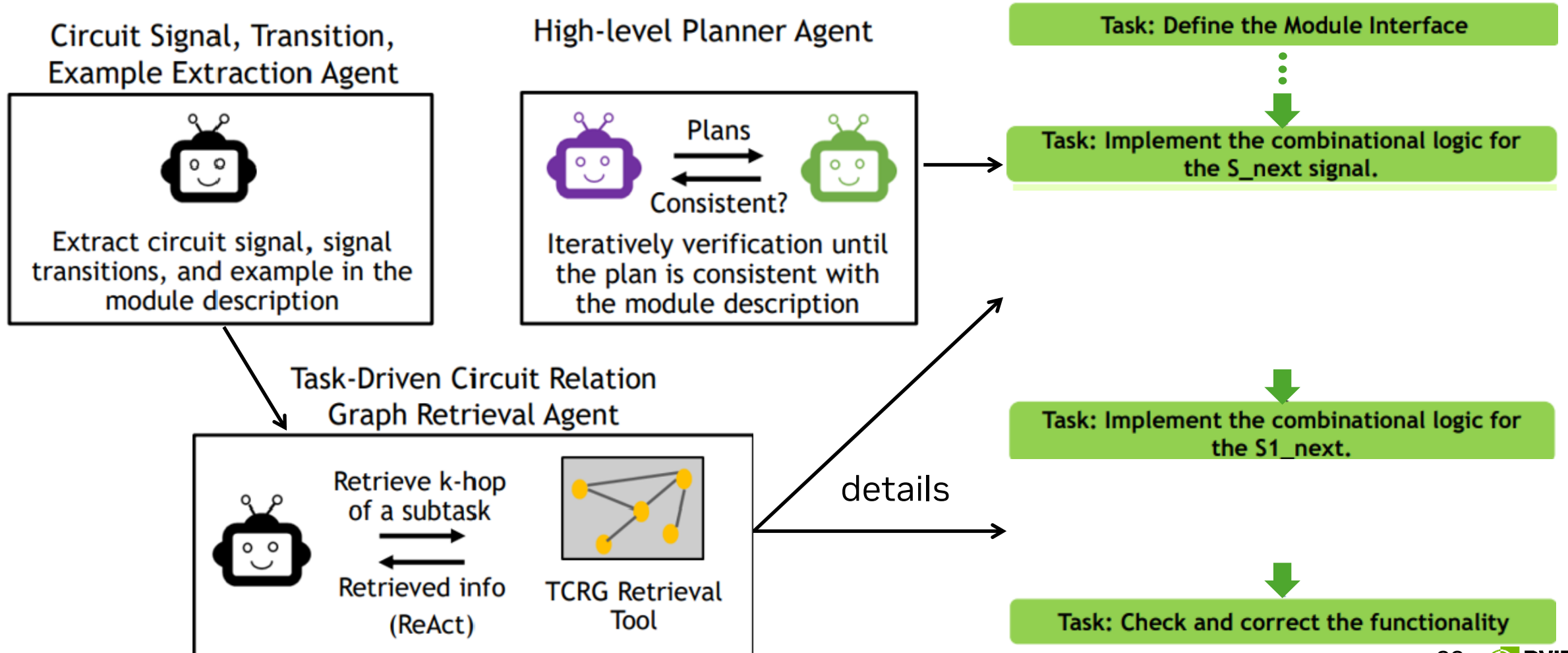
Task-Driven Circuit Relation Graph (TCRG)

Code Implementation

AST-guided waveform debugging tool



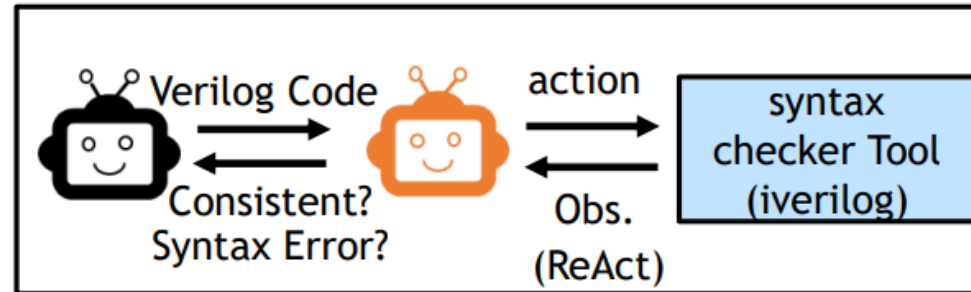
Task Planning



Code Implementation

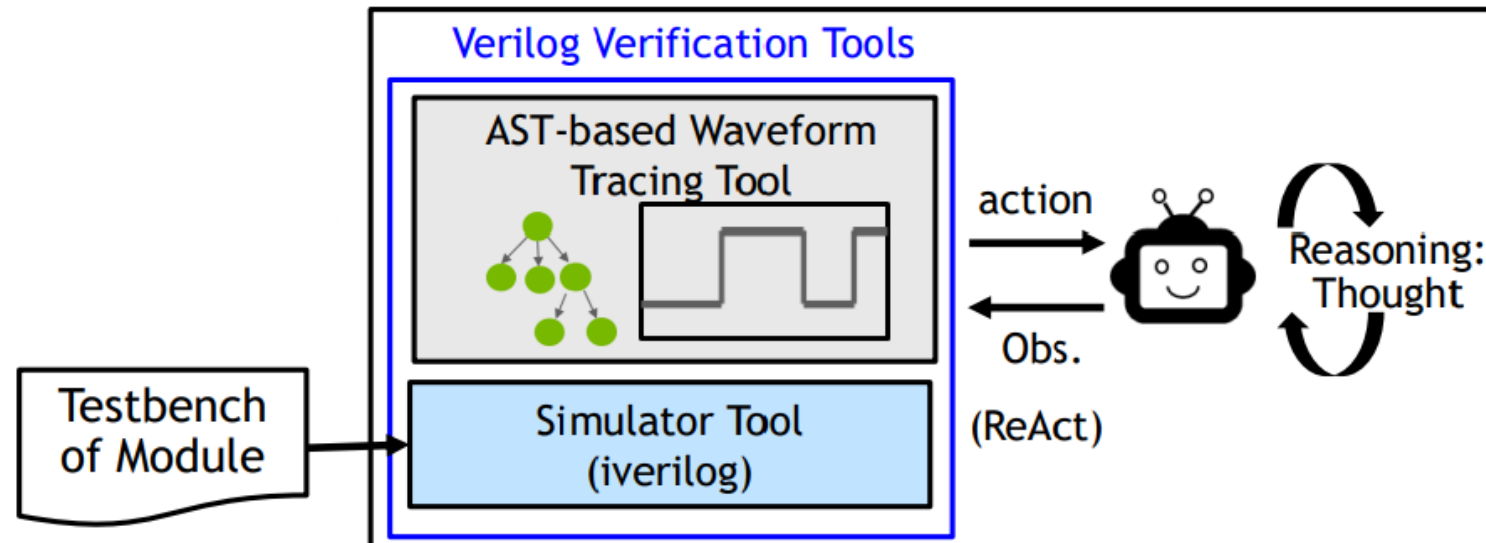
Code Agent: Write partial Verilog code

Writing code for each task



Debug Agent: Check and Correct the functionality

Debug final code



Future – AI Learning the Algorithm

Learning to decompose a problem into steps and beyond?

Write a bash script that takes a matrix represented as a string with format '[1,2],[3,4],[5,6]' and prints the transpose in the same format.

Formulating the problem

Crafting the script

Reading input and parsing

Breaking down the matrix

Determining matrix dimensions

Mapping out the matrix

Constructing the output

```
bash
Copy code

#!/bin/bash

INPUT="$1"

# Remove '[' and ']' and replace ',[' with newline
rows=$(echo "$INPUT" | sed 's/\],\[/\n/g' | sed 's/\[/\n/g' | sed 's/\]/\n/g')

# Read lines into array
mapfile -t lines <<< "$rows"

nrows=${#lines[@]}

# Get number of columns from the first line
cols=$(echo "${lines[0]}" | awk -F',' '{print NF}')

declare -a matrix

# Build the matrix
for ((i=0; i<nrows; i++)); do
    IFS=',' read -ra elements <<< "${lines[i]}"
    for ((j=0; j<${#elements[@]}; j++)); do
        index=$((i * cols + j))
        matrix[$index]=${elements[j]}
    done
done
```

“It learns to break down tricky steps into simpler ones. It learns to try a different approach when the current one isn’t working. This process dramatically improves the model’s ability to reason.”

OpenAI, Learning to Reason with LLMs

Closing Thoughts

Meet the challenges of AI systems with deep integration of AI and Algorithms

Leverage the advantages and avoid the pitfalls of either one

Explore the vast potential of integrating AI with algorithms for EDA problems: design analysis and optimization.

Exploit the revolutionary advancement of the Agentic system (an algorithm using LLM as a computing node) for design assistance

Algorithms designed by human based on their domain knowledge; but could be learned by AI in the future.

